| | |
|---|---|
| **Status** | Finished |
| **Started** | Monday, 28 September 2026, 11:27 AM |
| **Completed** | Friday, 9 October 2026, 10:55 PM |
| **Duration** | 11 days 11 hours |
| **Grade** | **10.00** out of 10.00 (**100**%) |

Question **1**

Correct

Mark 1.00 out of 1.00

Given a **non-empty** string s and an abbreviation abbr, return whether the string matches with the given abbreviation.

A string such as "word" contains only the following valid abbreviations:

["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1", "1o2", "2r1", "3d", "w3", "4"]

Notice that only the above abbreviations are valid abbreviations of the string "word". Any other string is not a valid abbreviation of "word".

**Note:**

Assume s contains only lowercase letters and abbr contains only lowercase letters and digits.

**Example 1:**

**Input**

internationalization
i12iz4n

**Output**

true

**Explanation**

Given **s** = "internationalization", **abbr** = "i12iz4n":

Return true.

**Example 2:**

**Input**

apple
a2e

**Output**

false

**Explanation**

Given **s** = "apple", **abbr** = "a2e":

Return false.

**Answer:** (penalty regime: 0 %)

```
def validWordAbbreviation(s, abbr):
    i = 0
    j = 0

    while i < len(s) and j < len(abbr):
        if abbr[j].isdigit():
            if abbr[j] == '0':
                return False

            num = 0
            while j < len(abbr) and abbr[j].isdigit():
                num = num * 10 + int(abbr[j])
                j += 1

            i += num
        else:
            if s[i] != abbr[j]:
                return False
            i += 1
            j += 1

    return i == len(s) and j == len(abbr)


s = input().strip()
abbr = input().strip()

print(str(validWordAbbreviation(s, abbr)).lower())
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | internationalization<br>i12iz4n | true | true | ✓ |
| ✓ | apple<br>a2e | false | false | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input:

thequickbrownfoxjumpsoverthelazydog

Output:

true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input:

arvijayakumar

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

**For example:**

| Test | Result |
|---|---|
| `print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))` | `true` |
| `print(checkPangram('arvijayakumar'))` | `false` |

**Answer:** (penalty regime: 0 %)

Reset answer

```
def checkPangram(sentence: str) -> str:
    # Check if the number of unique characters is 26
    if len(set(sentence)) == 26:
        return 'true'
    else:
        return 'false'

```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | `print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))` | `true` | `true` | ✓ |

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | `print(checkPangram('arvijayakumar'))` | `false` | `false` | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

The program must accept **N** series of keystrokes as string values as the input. The character **^** represents undo action to clear the last entered keystroke. The program must print the string typed after applying the undo operations as the output. If there are no characters in the string then print **-1** as the output.

**Boundary Condition(s):**

1 <= N <= 100
1 <= Length of each string <= 100

**Input Format:**

The first line contains the integer N.
The next N lines contain a string on each line.

**Output Format:**

The first N lines contain the string after applying the undo operations.

**Example Input/Output 1:**

Input:

3
Hey ^ goooo^^glee^
lucke^y ^charr^ms
ora^^nge^^^^

Output:

Hey google
luckycharms
-1

**Answer:** (penalty regime: 0 %)

```
import sys

def solve():
    # Read all lines from standard input
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    # First line contains the integer N
    n = int(input_data[0].strip())

    # Process each of the next N lines
    for i in range(1, n + 1):
        if i >= len(input_data):
            break
        s = input_data[i]

        stack = []
        for char in s:
            if char == '^':
                if stack:
                    stack.pop()  # Undo: remove the last entered character
```

```
            else:
                stack.append(char)

        # Join the stack characters to form the final string
        result = "".join(stack)

        # If the string is empty, print -1; otherwise, print the string
        if len(result) == 0:
            print("-1")
        else:
            print(result)

if __name__ == "__main__":
    solve()

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 3<br>Hey ^ goooo^^glee^<br>lucke^y ^charr^ms<br>ora^^nge^^^^ | Hey google<br>luckycharms<br>-1 | Hey google<br>luckycharms<br>-1 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Assume that the given string has enough memory.

Don't use any extra space(IN-PLACE)

**Sample Input 1**

a2b4c6

**Sample Output 1**

aabbbbccccccc

**Answer:** (penalty regime: 0 %)

```
s = list(input().strip())

i = 0
while i < len(s):
    if s[i].isdigit():
        j = i
        while j < len(s) and s[j].isdigit():
            j += 1

        count = int(''.join(s[i:j]))
        char = s[i - 1]

        s[i - 1:j] = [char] * count
        i = i - 1 + count
    else:
        i += 1

print(''.join(s))
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | a2b4c6 | aabbbbccccccc | aabbbbccccccc | ✓ |
| ✓ | a12b3d4 | aaaaaaaaaaaabbbdddd | aaaaaaaaaaaabbbdddd | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

**Note:** For the purpose of this problem, we define empty string as valid palindrome.

**Example 1:**

```
Input:
A man, a plan, a canal: Panama

Output:
1
```

**Example 2:**

```
Input:
race a car

Output:
0
```

**Constraints:**

- `s` consists only of printable ASCII characters.

**Answer:** (penalty regime: 0 %)

```
import sys

def main():
    # Read all contents from stdin (handles single or multi-line inputs)
    s = sys.stdin.read()

    # Filter only alphanumeric characters and convert to lowercase
    cleaned = [ch.lower() for ch in s if ch.isalnum()]

    # Check if the cleaned list reads the same forward and backward
    if cleaned == cleaned[::-1]:
        print(1)
    else:
        print(0)

if __name__ == "__main__":
    main()
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | A man, a plan, a canal: Panama | 1 | 1 | ✓ |
| ✓ | race a car | 0 | 0 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given a string S which is of the format USERNAME@DOMAIN.EXTENSION, the program must print the EXTENSION, DOMAIN, USERNAME in the reverse order.

**Input Format:**

The first line contains S.

**Output Format:**

The first line contains EXTENSION.
The second line contains DOMAIN.
The third line contains USERNAME.

**Boundary Condition:**

1 <= Length of S <= 100

**Example Input/Output 1:**

Input:

abcd@gmail.com

Output:

com
gmail
abcd

**For example:**

| Input | Result |
|---|---|
| arvijayakumar@rajalakshmi.edu.in | edu.in<br>rajalakshmi<br>arvijayakumar |

**Answer:** (penalty regime: 0 %)

```
import sys

def process_email():
    # Read the full input line and strip any trailing newlines/spaces
    s = sys.stdin.read().strip()
    if not s:
        return

    # Split into username and the rest of the domain/extension part
    username, rest = s.split('@', 1)

    # Split the remaining part by the first dot to separate domain and extension
    domain, extension = rest.split('.', 1)

    # Print the values in the requested reverse order
    print(extension)
    print(domain)
    print(username)

if __name__ == "__main__":
    process_email()

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | abcd@gmail.com | com<br>gmail<br>abcd | com<br>gmail<br>abcd | ✓ |
| ✓ | arvijayakumar@rajalakshmi.edu.in | edu.in<br>rajalakshmi<br>arvijayakumar | edu.in<br>rajalakshmi<br>arvijayakumar | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Consider the below words as key words and check the given input is key word or not.

keywords: {break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var}

Input format:

Take string as an input from stdin.

Output format:

Print the word is key word or not.

Example Input:

break

Output:

break is a keyword

Example Input:

IF

Output:

IF is not a keyword

**For example:**

| Input | Result |
|---|---|
| break | break is a keyword |
| IF | IF is not a keyword |

**Answer:** (penalty regime: 0 %)

```
import sys

def check_keyword():
    # Define the set of valid, case-sensitive keywords
    keywords = {
        "break", "case", "continue", "default", "defer", "else",
        "for", "func", "goto", "if", "map", "range",
        "return", "struct", "type", "var"
    }

    # Read the word from standard input and clean trailing newlines/spaces
    word = sys.stdin.read().strip()

    # Check if the word exists in the keyword set
    if word in keywords:
        print(f"{word} is a keyword")
    else:
        print(f"{word} is not a keyword")

if __name__ == "__main__":
    check_keyword()

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | break | break is a keyword | break is a keyword | ✓ |
| ✓ | IF | IF is not a keyword | IF is not a keyword | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Write a Python program to get one string and reverses a string. The input string is given as an array of characters `char[]`.

You may assume all the characters consist of printable ascii characters.

**Example 1:**

```
Input:
hello
Output:
olleh
```

**Example 2:**

```
Input:
Hannah
Output:
hannaH
```

**Answer:** (penalty regime: 0 %)

```
import sys

def reverseString(s: list) -> None:
    # Use two pointers to swap elements in-place
    left, right = 0, len(s) - 1
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1

if __name__ == "__main__":
    # Read the full string from input, stripping trailing newlines/carriage returns
    input_str = sys.stdin.read().strip('\r\n')

    # Convert string into an array (list) of characters
    char_array = list(input_str)

    # Reverse the array in-place
    reverseString(char_array)

    # Print the resulting reversed string
    print("".join(char_array))

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | hello | olleh | olleh | ✓ |
| ✓ | Hannah | hannaH | hannaH | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Find if a String2 is substring of String1. If it is, return the index of the first occurrence. else return -1.

**Sample Input 1**

thistest123string

123

**Sample Output 1**

8

**Answer:** (penalty regime: 0 %)

```
def findSubstringIndex(string1: str, string2: str) -> int:
    # Use the native find method which returns the start index or -1 if not found
    return string1.find(string2)

# --- Handling standard competitive programming inputs ---
if __name__ == "__main__":
    import sys
    # Read all input data cleanly, removing any trailing newlines or spaces
    input_data = sys.stdin.read().splitlines()
    if len(input_data) >= 2:
        string1 = input_data[0].strip()
        string2 = input_data[1].strip()
        print(findSubstringIndex(string1, string2))

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | thistest123string<br>123 | 8 | 8 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Constraints:

1 <= s.length <= 10^4

s consists of parentheses only '()[]{}'.

**For example:**

| Test | Result |
|---|---|
| print(ValidParenthesis("()")) | true |
| print(ValidParenthesis("()[]{}")) | true |
| print(ValidParenthesis("(]")) | false |

**Answer:** (penalty regime: 0 %)

Reset answer

```
def ValidParenthesis(s: str) -> str:
    # Map closing brackets to their corresponding open brackets
    bracket_map = {')': '(', '}': '{', ']': '['}
    stack = []

    for char in s:
        # If it's a closing bracket
        if char in bracket_map:
            # Pop the top element if stack is not empty; otherwise assign a dummy value
            top_element = stack.pop() if stack else '#'

            # If the mapping doesn't match the popped element, it's invalid
            if bracket_map[char] != top_element:
                return 'false'
        else:
            # If it's an opening bracket, push it onto the stack
            stack.append(char)

    # If the stack is empty, all brackets were matched correctly
    return 'true' if not stack else 'false'

```

| | Test | Expected | Got | |
|---|---|---|---|---|
| ✓ | print(ValidParenthesis("()")) | true | true | ✓ |
| ✓ | print(ValidParenthesis("()[]{}")) | true | true | ✓ |
| ✓ | print(ValidParenthesis("(]")) | false | false | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.